

AI-Assisted Update Guide

Cell Salts & Astrology Tracker — Expo / React Native / TypeScript

How to hand this project to other AI coding tools, quickly and without breaking it.

✓ How to use this document

New to this project? Start at Section 1 (Quick Reference Card) — it points you to everything else. Handing work to another AI tool? Paste Section 4 (Context Primer) as your first message, then give it the matching recipe from Section 6 (Task Recipes) instead of a vague instruction — the recipes exist so the tool doesn't have to guess the project's conventions or rediscover a bug that's already been fixed once (Section 7). Always finish with Section 8 (Verification Checklist).

1. Quick Reference Card

The fastest path through this document. Find your situation below and go straight to that section — everything here is explained in full further in.

I need to...	Go to
Understand the whole project in under a minute	Section 2 + Section 3
Brief another AI tool before it touches any code	Section 4 (paste verbatim)
Add a screen, component, profile field, theme, salt/sign, package, or restyle something	Section 6 — find the matching recipe and hand it over as-is
Figure out which file actually owns a piece of behavior	Section 6 recipes each open with the owning file(s)
A bug/symptom looks familiar	Section 7 — check before re-diagnosing from scratch
Confirm a change is actually safe to keep	Section 8 — run every step, every time
Add or check an exact package version	Section 9
A change won't resolve after a couple of tries	Section 10

The three rules that get broken most often

- 1 One of everything.** One navigation system (React Navigation — never Expo Router), one state store (`src/store/useAppStore.ts`), one design-token file (`src/constants/theme.ts`). If a tool wants to add a second one "for best practice," it's wrong for this project.
- 2 Zustand selectors must return a stable reference.** Never `useAppStore(s => s.logs.slice(0,3))` inline — that returns a new array every render and causes an infinite update loop. Select the raw field, derive with `useMemo` in the component.

- 3 Colors always come from** `useTheme()`. Never a hardcoded hex value. Interactive/brand color → accent. Neutral surface (card, input background) → `colors.card` / `colors.surfaceAlt` (already gradient-tinted in dark mode). Semantic → `colors.success` / `colors.danger` / etc. There is no `colors.primary` anymore — an older version of this app had one, fixed to navy blue regardless of theme, which was itself a bug that got fixed (Section 7).

2. Project Snapshot

Layer	Choice
Framework	Expo SDK 57 (React Native 0.86, React 19.2.3)
Language	TypeScript, strict mode (tsconfig extends expo/tsconfig.base)
Navigation	React Navigation v7 — bottom-tabs + native-stack. Entry: index.ts → App.tsx. 4 tabs: Home, Tracker, Analytics, Profile. This project does NOT use Expo Router and does NOT have a Directory tab (removed — see Section 7).
State	Zustand single store (src/store/useAppStore.ts), persisted to AsyncStorage
UI system	Glassmorphic — expo-blur BlurView cards floating over a full-screen expo-linear-gradient backdrop (6 selectable presets). Every preset also carries an accent color and tinted dark-mode surface colors — see useTheme() in Section 3.
Storage/Export	AsyncStorage (state) + expo-file-system's new File/Paths API + expo-print/expo-sharing (JSON/PDF export)
Notifications	expo-notifications (local, scheduled follow-up reminders)
Astrology math	src/utils/astronomy.ts — real (not simulated) low-precision formulas for Moon Sign and Ascendant, independently verified — see Section 7. Computed AUTOMATICALLY (a useEffect in ProfileScreen) once birth date/time/location are all set — no manual "Calculate" button, no manual picker for Moon Sign or Ascendant (only Sun Sign has one).

■ Single most important fact

The bundler will sometimes print "Using src/app as the root directory for Expo Router." if expo-router happens to be installed as a dependency somewhere in the tree. Ignore that message unless you have deliberately migrated this app to file-based routing — the real entry point is index.ts → App.tsx → RootNavigator, confirmed by the bundler's own "Import stack" trace on any error.

3. Folder Map

Every file an AI tool should know about before making changes:

App.tsx	Entry point: ErrorBoundary > GestureHandlerRootView > SafeAreaProvider > GradientBackdrop + RootNavigator + DisclaimerModal
app.json	Expo config (icons, permissions, plugin entries for expo-notifications and expo-location)
src/	
components/	
BarChart.tsx	Dependency-free bar chart (analytics)
Basics.tsx	SectionHeader, Chip (a "button" - see Section 7), EmptyState
Button.tsx	Primary/secondary/outline/danger button, theme-aware
Card.tsx	THE glass card (BlurView+tint) - reuse it, don't reinvent
CellSaltListItem.tsx	A tappable salt row (used in Find My Salt's browse list)
DateInput.tsx	Typed MM/DD/YYYY field + native picker modal
DisclaimerModal.tsx	First-launch medical disclaimer
ErrorBoundary.tsx	Top-level crash fallback (class component)
GradientBackdrop.tsx	Full-screen animated gradient (renders once in App.tsx)
SeverityScale.tsx	1-10 severity picker
StarRating.tsx	5-star input
TimeInput.tsx	Typed HH:MM field + native picker modal (mirrors DateInput)
ZodiacBadge.tsx	Zodiac glyph badge
constants/	
cellSalts.ts	THE 12 cell salts dataset + lookup helpers - single source
theme.ts	LIGHT/DARK_THEME, spacing/radius/font tokens, GRADIENT_THEMES presets (gradient stops + accent colors + dark-mode surface tints) - the single design-token file
zodiac.ts	THE 12 zodiac signs + getZodiacSignForDate() - single source
navigation/	
MainTabNavigator.tsx	Bottom tabs (animation: 'shift', blurred tab bar + header)
RootNavigator.tsx	Stack wrapping the tabs + modal/detail screens
types.ts	RootStackParamList / MainTabParamList - add new screens here
screens/	One screen per file, named <Name>Screen.tsx. No DirectoryScreen - salt browsing lives inside FindMySaltScreen's "By Symptom" mode.
store/	
useAppStore.ts	THE single Zustand store (profile + logs), persisted
types/	
index.ts	THE single source of truth for domain types (ZodiacSign, CellSalt, LogEntry, UserProfile, etc.)
utils/	
astronomy.ts	Moon Sign / Ascendant calculations - verified formulas, see Section 7 before "simplifying" any of the math
dateUtils.ts	Formatting helpers
exportUtils.ts	JSON/PDF export (uses expo-file-system File/Paths)
notifications.ts	Local notification scheduling
useTheme.ts	Hook: colors (theme + dark-mode surface tint), accent (theme-aware brand color), gradient, isDark

■ Installed but unused

expo-constants, expo-device, and react-native-svg are in package.json but not imported anywhere in src/ (leftover from earlier experiments). Don't assume something must already use them — check with grep before building on top of them, and feel free to remove them if you want a leaner dependency list.

4. Context Primer — Paste This First

Copy the block below as your **first message** to any other AI tool before asking it to change code. It gives the tool the constraints it needs to avoid inventing a conflicting pattern (a second state store, a duplicate theme file, Expo Router pages, a rebuilt Directory screen, etc.).

You are editing an existing Expo (SDK 57) + React Native + TypeScript app. Follow these constraints exactly -- do not deviate even if you think a different pattern is "better practice" in general:

1. Navigation is React Navigation v7 (bottom-tabs + native-stack), entry via `index.ts -> App.tsx -> src/navigation/RootNavigator.tsx`. Do NOT introduce Expo Router or a `src/app/` routing directory. There are exactly 4 tabs (Home, Tracker, Analytics, Profile) -- there is no Directory tab; salt browsing/search lives inside `FindMySaltScreen`'s "By Symptom" mode.
2. State lives in exactly one Zustand store: `src/store/useAppStore.ts`, persisted to `AsyncStorage`. Do not create a second store or move state into `React Context/Redux`.
3. All domain types live in `src/types/index.ts`. Extend that file; don't redefine types locally in a screen/component.
4. All design tokens (colors, spacing, radius, font sizes, gradient presets, accent colors, dark-mode surface tints) live in `src/constants/theme.ts`. Don't hardcode hex colors or pixel values in components -- import from `theme.ts`, and read the resolved values via the `useTheme()` hook (colors, accent, gradient, `isDark`).
5. Every card/panel uses the existing `<Card>` component (`src/components/Card.tsx`), which renders a blurred, translucent "glass" surface over the app's gradient background. Reuse `<Card>` rather than writing new `BlurView` usage elsewhere. On Android, `BlurView` here deliberately uses the default `blurMethod` ('none' -- a flat semi-transparent view, not a true blur): real Android blur (`BlurTargetView + blurMethod="dimezisBlurViewSdk31Plus"`) was tried and crashed the app on any touch on-device. Don't re-attempt it without a real crash log and testing one change at a time (see known-gotchas doc).
6. Screen root containers use `backgroundColor: 'transparent'` (never a solid color) so the global gradient (rendered once in `App.tsx` via `GradientBackdrop`) shows through.
7. Any "brand"/interactive color (buttons, selected chips, active tab, nav tint) must use ``accent`` from `useTheme()` -- never a fixed color, and never `colors.primary` (that field doesn't exist; using a fixed color there was itself a past bug -- see the known-gotchas doc).

8. Read state from Zustand with a selector that returns a STABLE reference. Never do ``useAppStore(s => s.someArray.map(...))`` or ``.filter(...).slice(...)`` inline in a selector -- that returns a new array every render and causes an infinite re-render loop ("Maximum update depth exceeded" / "getSnapshot should be cached"). Instead: select the raw field, then derive with `useMemo` in the component.
9. Any new package must be added with ``npx expo install <package>``, not ``npm install <package>`` -- this resolves to the version compatible with this project's Expo SDK (57). If a package needs native code and the project is only tested in Expo Go, flag that a custom dev client / prebuild may be required.
10. `src/utils/exportUtils.ts` uses `expo-file-system`'s `NEW File/Paths` class API (SDK 54+), not the deprecated `FileSystem.documentDirectory` string API. Keep using the new API in any related changes.
11. `src/utils/astronomy.ts` implements real astronomical formulas (Moon position, Ascendant) that were independently verified against a published worked example and first-principles checks. Don't rewrite or "simplify" this math from memory -- if it must change, re-derive or re-verify it the same way (see the project's known-gotchas doc) before trusting the output.
12. Moon Sign and Ascendant are calculated AUTOMATICALLY in `ProfileScreen.tsx` (a `useEffect` that fires once birth date, birth time, latitude, longitude, and UTC offset are all present) and written to the store only when the computed value actually differs from what's stored. There is no "Calculate" button and no manual Chip picker for Moon Sign/Ascendant -- only Sun Sign has one. Don't reintroduce either.
13. When you (the human) receive an updated project archive after a change that removed or renamed files, replace the whole project folder rather than extracting the new zip on top of the old one -- `unzip` never deletes files that aren't in the new archive, and ``tsc`` checks every `.tsx` file it finds, so an orphaned file from a removed feature will still throw compile errors even though nothing imports it.
14. Run ``npx tsc --noEmit`` after every change and fix all errors before considering the change done. If possible, also run ``npx expo export --platform android`` (and ``--platform web`` if the change touches anything platform-sensitive) to confirm the bundler can build the whole module graph with no "Unable to resolve" errors.

Now here is the specific change I want: <describe your change here>

5. Golden Rules — Do / Don't

Do

- Ask for **one scoped change at a time** ("add a filter to the Tracker list") rather than open-ended requests ("improve the app") — or better, hand over the matching recipe from Section 6 verbatim.

- Ask the tool to show a **full-file rewrite** (not a fragile inline patch) whenever it touches a shared file: `theme.ts`, `useAppStore.ts`, `types/index.ts`, or any navigator.
- Ask it to **select stable references from Zustand** and derive filtered/sliced/mapped views with `useMemo` in the component body.
- Ask it to reuse `<Card>`, `<Button>`, `<Chip>`, and the tokens in `theme.ts` instead of inventing new styling primitives.
- Ask it to run **`npx expo install`** for any new native/Expo package, and to tell you if the package requires a custom dev client instead of Expo Go.
- Commit to git (or copy the project folder) **before** a large AI-driven change, so you can diff or roll back — the ZodiacWheel experiment in Section 7 was only safe to remove cleanly because it was scoped to a few isolated files.
- **Fully replace the project folder** when you receive an updated zip/archive after a change that removed or renamed files — delete the old folder first, or diff the archive's file list against what's on disk. Extracting on top of an existing folder only adds/overwrites files; it never deletes ones that were removed, leaving orphaned files behind (see Section 7).

Don't

- Don't let a tool add Expo Router, React Query, Redux, MobX, or a second navigation/state layer "for best practice" — this app intentionally uses one navigation library and one store.
- Don't let a tool rename or move files across `src/components`, `src/screens`, `src/navigation`, `src/store`, `src/types`, `src/utls` without also updating every import — ask it to grep for all usages first.
- Don't accept a Zustand selector that returns a freshly-created array/object every call (see Context Primer, rule 8) — this is the exact bug that caused the "Maximum update depth exceeded" crash during development.
- Don't let a tool bump `react`, `react-native`, or `expo` to a different major/minor version without also updating every peer dependency (`react-dom`, `react-native-web`, etc.) to a matching version.
- Don't accept solid `backgroundColor` values on screen containers — that silently hides the gradient backdrop everywhere below that screen.
- Don't let a tool recreate a `Directory/browse` screen from scratch — that functionality already exists inside `FindMySaltScreen`'s "By Symptom" mode; extend that instead.
- Don't let a tool re-add a manual Moon Sign/Ascendant picker or a "Calculate" button in `ProfileScreen` — both were deliberately removed in favor of automatic calculation (see Section 2 and Section 7).
- Don't let a tool set a non-default `blurMethod` on a `BlurView`, or add `BlurTargetView/freezeOnBlur`, without an actual device to test on and a way to pull a crash log first — that exact combination crashed the app on any touch when it was tried (Section 7).

6. Task Recipes for AI Tools

Hand the matching recipe to whichever AI tool is making the change, instead of a vague instruction. Each one names the exact file(s) to touch and the pattern to follow, so the tool doesn't have to guess this project's conventions — or rediscover, the hard way, a design decision that was already made on purpose.

A. Add a new screen

- 1 Create `src/screens/<Name>Screen.tsx`. Copy the shape of an existing screen of the same kind: a plain-content screen like `TrackerScreen.tsx`, or a search/filter/list screen like `FindMySaltScreen.tsx`'s "By Symptom" mode if it needs a growing list (fixed header block + a `flex:1 FlatList`, not a `FlatList` nested in a `ScrollView`).
- 2 Pull theme values from `useTheme()` (colors, accent) — never hardcode colors.
- 3 If it's a tab-root screen: add it to `MainTabParamList` in `src/navigation/types.ts`, then add a `<Tab.Screen>` entry + a glyph in `TAB_ICONS` inside `src/navigation/MainTabNavigator.tsx`.
- 4 If it's a pushed or modal detail screen instead: add it to `RootStackParamList` in `src/navigation/types.ts` with its param shape, then add a `<Stack.Screen>` entry in `src/navigation/RootNavigator.tsx` (set `presentation: 'modal'` if it should slide up over everything, like `FindMySalt` and `LogEntry` do).
- 5 Navigate to it from elsewhere with the same typed `navigation.navigate('<Name>', {...params})` pattern every existing screen already uses.

Done when: `npx tsc --noEmit` is clean, and the screen is reachable by tapping through the app on both a tab bar icon (if applicable) and any button/link that opens it.

B. Add a new reusable UI component

- 1 Create `src/components/<Name>.tsx`.
- 2 Pull colors from `useTheme()` (colors, accent) — never a hardcoded hex value.
- 3 Use `SPACING` / `RADIUS` / `FONT_SIZES` from `src/constants/theme.ts` for all spacing and sizing, not raw numbers.
- 4 If it's a "panel"/"card"-like surface, wrap it in the existing `<Card>` component instead of adding a new `BlurView` elsewhere.

Done when: it renders correctly in both light and dark mode, and with at least two different gradient presets selected (e.g. `Nebula` and `Sunset`) — nothing should look invisible or the same regardless of theme.

C. Add a field to the user profile / Zustand store

- 1 Add the field to the `UserProfile` interface in `src/types/index.ts` (mark it optional with `?` unless every existing stored profile will always have a value).
- 2 If it isn't optional, add a sane default to `DEFAULT_PROFILE` in `src/store/useAppStore.ts`.
- 3 Read/write it via the existing `updateProfile()` action — don't add a new setter unless the update logic is non-trivial (`setGradientTheme` is the example of when a dedicated action earns its place).
- 4 Any UI for setting it belongs in `ProfileScreen.tsx`, alongside the other profile fields, following the existing Card-per-setting layout.

Done when: the field round-trips through `AsyncStorage` — set it, fully reload the app, confirm the value is still there.

D. Add a new gradient theme preset

- 1 In `src/constants/theme.ts`, add the new id to the `GradientThemeId` union type and a matching entry to `GRADIENT_THEMES`, following the exact shape of an existing preset: `light/dark` gradient stops, `accentLight/accentDark`, and `darkSurface/darkSurfaceAlt/darkCard`.
- 2 Add the new id to `GRADIENT_THEME_ORDER` so it appears in the Profile picker.
- 3 Check contrast before committing any color — don't eyeball it. `accentLight` needs $\geq 4.5:1$ contrast against white text; `accentDark` needs $\geq 4.5:1$ against dark-navy (`#0B0E23`) text. Compute this with the WCAG relative-luminance formula (the existing 6 presets were verified this way, catching two that needed darkening).

Done when: the new swatch appears in Profile, and selecting it doesn't make any button, chip, or text illegible in either light or dark mode.

E. Add or edit a cell salt / zodiac sign entry

- 1 Cell salt data lives in `src/constants/cellSalts.ts` (the `CELL_SALTS` array). A new salt needs a unique id added to the `CellSaltId` union in `src/types/index.ts` first.
- 2 Zodiac data lives in `src/constants/zodiac.ts` (the `ZODIAC_SIGNS` array). Each sign maps to exactly one `cellSaltId`.
- 3 Don't hand-maintain a second copy of this data anywhere — every screen (`Find My Salt's` browse list, `CellSaltDetailScreen`, `Home`, `Analytics`) reads from these two arrays and their `*_MAP` lookup objects built from them.

Done when: the new/edited salt shows up correctly in Find My Salt's "By Symptom" list and its own detail screen, and (if new) in the Tracker/Analytics salt pickers, which are also sourced from `CELL_SALTS` automatically.

F. Add a new npm package

- 1 Run `npx expo install <package-name>` from the project root — never plain `npm install` for a package with native code, since `expo install` resolves the version actually compatible with this project's Expo SDK.
- 2 Check the package's Expo docs page for any required `app.json` config — `expo-notifications` and `expo-location` in this project are both examples of packages that need a `plugins[]` entry (see `app.json`).
- 3 If it has native code and the project is only being tested in Expo Go, say so explicitly — it may require `npx expo prebuild / a custom dev client`.

Done when: `npx tsc --noEmit` and `npx expo export --platform android` both stay clean after the install.

G. Change a color or restyle something

- 1 Never hardcode a hex color in a component. Interactive/"brand" color (buttons, selected chips, active tab) → `accent` from `useTheme()`. Neutral surface (card, input background) → `colors.card` / `colors.surfaceAlt` (already gradient-tinted in dark mode). Semantic color → the matching `colors.success` / `colors.warning` / `colors.danger` / `colors.gold`.
- 2 If a color genuinely needs to change for every theme, edit it once in `src/constants/theme.ts` (`LIGHT_THEME` / `DARK_THEME` / the relevant `GRADIENT_THEMES` fields) rather than overriding it locally in one component.

Done when: the change is visible correctly in both light and dark mode, across at least two gradient presets, without reducing text/icon contrast below readable (see Recipe D's contrast note — the same bar applies here).

7. Known Gotchas Already Hit & Fixed

If a symptom below reappears after an AI-driven change, it's a regression of a bug already solved once — point the tool at the fix.

Symptom	Root Cause	Fix
"Maximum update depth exceeded" / "getSnapshot should be cached" on web, crashes the whole nav tree	A Zustand selector returned a new array each render, e.g. <code>useAppState(s => s.logs.slice(0,3))</code>	Select the raw array (<code>useAppState(s => s.logs)</code>) and derive with <code>useMemo</code> in the component.
List rows overlap on phone-sized viewports (search/filter screens)	The results FlatList had no <code>flex:1</code> , and filter chips lived in nested horizontal FlatLists inside a plain column	Give the results FlatList <code>flex:1</code> ; replace nested FlatLists with plain horizontal ScrollViews for the small, non-virtualized filter rows; wrap the screen in <code>SafeAreaView</code> .
"Unable to resolve expo-blur" (or any other package) when running in an existing project	Only the <code>src/</code> folder was copied into another Expo project; the <code>package.json</code> dependencies never came along	Run <code>npx expo install</code> for every package this project's source actually imports (Section 9).
Bundler prints "Using src/app as the root directory for Expo Router" but the app still works	<code>expo-router</code> is present as a dependency somewhere in the tree (often from a starter template) but isn't actually the active entry	Harmless if <code>index.ts -> App.tsx</code> is confirmed in the bundler's own error "Import stack". Only a real issue if you intended to migrate to file-based routing.
Buttons/chips stay the same blue no matter which gradient theme is selected	Button/Chip/tab-tint read a fixed <code>colors.primary</code> constant, independent of the selected gradient preset	<code>colors.primary</code> was removed. Everything brand-colored now reads accent from <code>useTheme()</code> , which resolves per-preset (Recipe G).
Dark-mode surfaces (input fields, unselected chips, list rows) all look like the same fixed navy blue	<code>colors.surfaceAlt/card</code> were static <code>DARK_THEME</code> constants, not tinted per gradient preset (only blurred Cards adapted, via blurring the gradient itself)	<code>useTheme()</code> now merges in each preset's <code>darkSurface/darkSurfaceAlt/darkCard</code> (Recipe D) — verified $\geq 13:1$ text contrast before shipping.
Back button sometimes hard to see against the header, especially in dark mode	Header used a flat translucent color with no real backing layer, and the back-chevron tint wasn't guaranteed high-contrast	Header now uses a real <code>BlurView</code> background (like the tab bar) and <code>headerTintColor = accent</code> , which is contrast-checked, not <code>colors.text</code> .
A "quick" astrology calculation gives a plausible-looking but wrong result (e.g. Ascendant off by $\sim 180^\circ$)	Formulas recalled from memory can have subtle sign/argument errors that still "look" like a valid answer	<code>astronomy.ts</code> was checked against a published worked example AND a first-principles horizon check, which is what caught this exact bug — verify any math change the same way, don't just eyeball it.

Symptom	Root Cause	Fix
A whole experimental feature (e.g. a home-screen visualization) needs to come back out cleanly	N/A — this is about scoping, not a bug	The ZodiacWheel feature was removable in one pass specifically because it lived in its own component file plus a few isolated edits — keep new features that scoped.
A rebuilt "browse all salts" screen/tab appears after a change	The Directory tab was intentionally removed — its functionality now lives inside FindMySaltScreen's "By Symptom" mode	Extend FindMySaltScreen instead of recreating a Directory screen (Recipe A, Context Primer rule 1).
tsc errors reference a component that nothing imports (e.g. an old ZodiacWheel.tsx / a removed export like ZODIAC_WHEEL_COLORS)	An updated project zip was extracted ON TOP OF an existing folder — unzip only adds/overwrites files, it never deletes files that were removed in the new version, and tsc checks every .tsx file it finds, not just imported ones	Delete the orphaned file(s). Going forward, delete the old folder before extracting a new archive, or diff file lists first (Golden Rules, Section 5).
A manual Moon Sign/Ascendant picker or a "Calculate" button reappears in ProfileScreen	Both were deliberately removed in favor of automatic calculation (a useEffect that fires once all birth-detail fields are present)	Only Sun Sign has a manual picker. Moon Sign/Ascendant should never need a button — restore the useEffect pattern instead (Section 2, Recipe C).
Glass surfaces (cards, header, tab bar) look visibly "off" on a real Android device — flatter/more washed out than intended, most noticeable in light mode	Android's BlurView defaults to blurMethod "none" (a flat semi-transparent view, not a real blur) unless given a BlurTargetView + non-default blurMethod	UNRESOLVED — currently accepted as-is; this is expected/documented Expo behavior, not a bug. A real fix was attempted and reverted; see the crash-warning row below.
On a real Android device, the old screen's content briefly overlaps the new one for a couple seconds after tapping a button that pushes/opens a screen (tab taps are unaffected)	Android's native-stack makes BOTH the incoming and outgoing screen semi-transparent during a push/modal transition; this app's screens are ALSO intentionally transparent (for the gradient), so both show through at once	UNRESOLVED — a fix (freezeOnBlur) was attempted alongside the row above and reverted together with it; see the crash-warning row below before retrying either.
(Read this before touching BlurView/navigation again) An AI tool adds BlurTargetView + a non- default blurMethod + freezeOnBlur to fix the two rows above	This exact combination was built and installed on a real Android device once already — it made the ENTIRE app crash on any touch (any button, any tab). All three shipped in one pass, so which one caused it was never isolated (no crash log available)	Reverted in full, back to plain BlurView with the Android default. If retrying: get a real adb logcat/crash log FIRST, and add only ONE of the three at a time, testing on-device after each (README "Attempted But Reverted").

8. Verification Checklist — Run After Every AI-Driven Change

- 1 **Type-check.** Run `npx tsc --noEmit` — must exit clean. This catches broken imports, wrong prop types, and typos immediately, before you ever open the app.
- 2 **Bundle-check (Android).** Run `npx expo export --platform android --output-dir /tmp/check` — confirms the whole module graph resolves (catches missing/renamed packages) without launching a device. Delete the output folder afterward.
- 3 **Bundle-check (Web),** if the change touches anything platform-sensitive (pickers, file export, blur, location):
`npx expo export --platform web --output-dir /tmp/check-web`.
- 4 **Run it for real.** `npx expo start`, then open on the platform(s) you actually ship to. Exercise the specific screen(s) the change touched.
- 5 **Watch the console for the known crash signatures** from Section 7 (infinite update loop / `getSnapshot` warning). If one appears, it's almost always a Zustand selector returning a new reference.
- 6 **Grep for duplicates** the tool might have introduced: a second `create(` (Zustand store), a second file exporting `ThemeColors`, a new `src/app/` directory, or a rebuilt `Directory` screen.
- 7 **Diff package.json.** Confirm no dependency's major version changed unless you asked for an upgrade, and that any new package was added via `npx expo install` (SDK-compatible range, not an arbitrary "latest").
- 8 **If the change touched colors,** check it in both light and dark mode across at least two gradient presets — see Recipe D/G's contrast note.

✓ **Quick copy-paste for steps 1-3**

```
npx tsc --noEmit && npx expo export --platform android --output-dir /tmp/check && npx expo export  
--platform web --output-dir /tmp/check-web && echo "ALL CLEAR" || echo "FIX ERRORS ABOVE BEFORE  
CONTINUING"
```

9. Dependency Reference

Exact versions this project was built and verified against. If another AI tool needs to add a package, install it with `npx expo install <name>` so it resolves to whatever range is correct for your project's actual installed Expo SDK (which may differ slightly from the versions below if time has passed).

Package	Version	Used for
@react-native-async-storage/async-storage	^3.1.1	Zustand persist storage backend
@react-native-community/datetimepicker	^9.2.1	Native picker inside DateTimePicker
@react-navigation/bottom-tabs	^7.18.18	Bottom tab navigator
@react-navigation/native	^7.3.18	Navigation container/core
@react-navigation/native-stack	^7.18.10	Stack navigator (detail/modal screens)
expo	~57.0.20	Framework
expo-blur	^57.0.2	Glass card / header / tab bar BlurView
expo-file-system	^57.0.6	JSON export (new File/Paths API)
expo-linear-gradient	^57.0.1	Gradient backdrop
expo-location	^57.0.16	"Use Current Location" shortcut for Ascendant calc
expo-notifications	^57.0.17	Local follow-up reminders
expo-print	^57.0.1	PDF export
expo-sharing	^57.0.18	Share sheet for exports
expo-status-bar	~57.0.1	Status bar styling
react	19.2.3	Core
react-dom	^19.2.3	Web target
react-native	0.86.3	Core
react-native-gesture-handler	^3.2.1	Required by navigation + App root
react-native-safe-area-context	^5.9.1	Safe-area insets
react-native-screens	^4.27.0	Native-stack/bottom-tabs dependency
react-native-web	^0.21.2	Web target
zustand	^5.0.15	State store

■ Installed but not actually imported

expo-constants, expo-device, and react-native-svg are present in package.json from earlier work but nothing in src/ currently imports them (confirm with grep before relying on that; verified true as of this revision). Safe to leave, or remove.

Dev dependencies

Package	Version
@types/react	~19.2.2
typescript	~6.0.3

npm scripts

```
npm start      expo start
npm run android expo start --android
npm run ios     expo start --ios
npm run web     expo start --web
```

10. If a Change Still Breaks Something

- 1 Copy the exact error text and the bundler's "Import stack" (if shown) back to the AI tool that made the change — don't paraphrase it.
- 2 If the tool can't resolve it in 1-2 tries, revert the change (git checkout / restore your pre-change copy) rather than letting the tool "keep trying" — accumulated speculative fixes compound into a worse mess.
- 3 Bring the reverted state, the original goal, and the error to a fresh AI session with the Context Primer (Section 4) attached, plus the specific recipe from Section 6 — a clean context with an explicit recipe often succeeds where an increasingly confused open-ended conversation won't.
- 4 For anything involving native modules (camera, background tasks, etc.) that doesn't work in Expo Go, confirm whether the project has moved to a custom dev client (npx expo prebuild) — that changes what "npx expo install" alone can fix.

Generated as a working reference for the Cell Salts & Astrology Tracker project. Keep this alongside the repo and update Section 7 (Known Gotchas) and Section 9 (Dependency Reference) whenever a new lesson or dependency is added — and add a new recipe to Section 6 whenever a task type comes up for the third time.